

Guía de Despliegue — PartsBot (FCH AutoLab)

Plataforma e-commerce B2B de autopartes con FastAPI + SQLAlchemy, frontend en vanilla JS, búsqueda bilingüe (ES/EN), integración con CassChoice y aplicación de margen del 6%.

1. Requisitos previos

- **Python 3.10+**
- **pip / venv**
- (Opcional para producción) **PostgreSQL 13+** y/o **Docker + Docker Compose**
- Credenciales de **CassChoice** (`CASS_SID` , `CASS_TOKEN`) para el sincronizador
- (Opcional) **GEMINI_API_KEY** para completar automáticamente marcas chinas con IA

2. Configuración desde cero

```
# 1. Clonar el repositorio
git clone https://github.com/fjcht/partsbot.git
cd partsbot

# 2. Crear y activar un entorno virtual
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Crear el archivo de configuración a partir del ejemplo
cp .env.example .env
```

Editar `.env`

Variable	Descripción	Valor recomendado
<code>DATABASE_URL</code>	Vacío = SQLite local. Para producción usar PostgreSQL	<code>postgresql+psycopg2://postgres:admin123@localhost:5432/repuestos_db</code>
<code>SECRET_KEY</code>	Secreto para firmar JWT	Genera uno: <code>python -c "import secrets; print(secrets.token_hex(32))"</code>
<code>JWT_ALGORITHM</code>	Algoritmo JWT	<code>HS256</code>
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Expiración del token	<code>1440</code>
<code>CASS_SID</code> / <code>CASS_TOKEN</code>	Sesión de CassChoice	(tus credenciales)
<code>CASS_BASE_URL</code>	Endpoint CassChoice	<code>https://merchant.casschoice.com</code>
<code>MARGEN_GANANCIA</code>	Margen sobre precio FOB	<code>0.06</code> (6%)
<code>GEMINI_API_KEY</code>	Clave IA (opcional)	(tu clave)

⚠ `.env` **NUNCA** se sube a Git (está en `.gitignore`). Solo se versiona `.env.example`.

Inicializar la base de datos

```
python init_db.py          # crea las 10 tablas + 55 traducciones ES/EN
python init_db.py --reset  # BORRA y recrea todo desde cero
```

Esto crea las tablas: `usuarios`, `catalogo_vehiculos`, `autopartes`, `compatibilidades`, `precios_casschoice`, `carritos`, `items_carrito`, `ordenes`, `items_orden`, `traducciones_partes`.

3. Ejecutar la aplicación

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

- **Frontend:** `http://localhost:8000/`
- **Health check:** `http://localhost:8000/api/health`
- **Docs Swagger:** `http://localhost:8000/docs`

Con Docker

```
docker-compose up --build
```

4. Ejecutar el sincronizador

El sincronizador importa vehículos y piezas desde CassChoice, aplica el margen del 6% y genera compatibilidades por rangos de años. Requiere `CASS_SID` y `CASS_TOKEN` en `.env`.

```
# Sincronización completa (vehículos + piezas)
python sincronizador.py

# Solo la fase de vehículos (Fase A)
python sincronizador.py --solo-vehiculos

# Solo la fase de piezas (Fase B)
python sincronizador.py --solo-piezas

# Sincronizar piezas concretas por número de parte
python sincronizador.py --piezas F4J16-3705110AB OTRO-CODIGO

# Sincronizar piezas desde un archivo (una por línea)
python sincronizador.py --archivo-piezas seed_parts.txt
```

También puede dispararse vía API: `POST /sincronizacion/...` (ver `/docs`).

5. Cómo probar cada funcionalidad

5.1 Servidor FastAPI

```
curl http://localhost:8000/api/health
# → {"status":"ok","version":"2.0.0","margen":0.06}
```

5.2 Base de datos

```
python init_db.py --reset
python -c "import sqlite3;c=sqlite3.connect('partsbot.db');print([r[0] for r in c.execute('SELECT name FROM sqlite_master WHERE type='table'\")])"
# → debe listar las 10 tablas
```

5.3 Frontend

Abrir `http://localhost:8000/` — debe cargar el catálogo B2B sin errores de consola.

Probar la búsqueda bilingüe escribiendo `pastillas de freno` o `brake pads`.

5.4 Autenticación (JWT)

```
# Registro
curl -X POST http://localhost:8000/auth/registro \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"secret123","nombre":"Test"}'

# Login → devuelve access_token
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"secret123"}'
```

5.5 Catálogo y búsqueda

```
curl http://localhost:8000/catalogo/marcas
curl "http://localhost:8000/consultar"
curl -X POST http://localhost:8000/busqueda/semantica \
  -H "Content-Type: application/json" \
  -d '{"termino":"pastillas de freno"}'
```

5.6 Carrito y órdenes (requieren token)

```
TOKEN="<access_token>"
curl -X POST http://localhost:8000/carrito/agregar \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"autoparte_id":1,"cantidad":1}'
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/carrito/ver
curl -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8000/ordenes/crear
```

6. Troubleshooting común

Problema	Causa probable	Solución
<code>ModuleNotFoundError</code>	Dependencias no instaladas / venv inactivo	<code>source .venv/bin/activate</code> && <code>pip install -r requirements.txt</code>
El servidor no arranca	Puerto 8000 ocupado	Usar otro puerto: <code>uvicorn main:app --port 8001</code>
<code>no such table</code>	BD no inicializada	Ejecutar <code>python init_db.py</code>
Frontend carga pero sin datos	BD vacía	Ejecutar el sincronizador o poblar datos de prueba
<code>404</code> en <code>/catalogo/marcas</code>	Ruta mal escrita	Las rutas NO llevan prefijo <code>/api</code> (excepto <code>/api/health</code>)
Error <code>401</code> en carrito/órdenes	Falta el token JWT	Iniciar sesión y enviar <code>Authorization: Bearer <token></code>
Sincronizador falla con <code>403/401</code>	<code>CASS_SID</code> / <code>CASS_TOKEN</code> expirados	Renovar credenciales en <code>.env</code>
Precios sin margen	<code>MARGEN_GANANCIA</code> mal configurado	Verificar <code>MARGEN_GANANCIA=0.06</code> en <code>.env</code>
IA no completa datos	Falta <code>GEMINI_API_KEY</code>	Añadir la clave en <code>.env</code> (funcionalidad opcional)
Cambios en <code>.env</code> no aplican	Servidor no reiniciado	Reiniciar uvicorn

7. Estructura del proyecto

```

partsbot/
├── main.py           # App FastAPI + montaje de routers + frontend
├── config.py         # Configuración vía variables de entorno
├── database.py       # Engine y sesión SQLAlchemy
├── models.py         # 10 modelos ORM (esquema normalizado)
├── schemas.py        # Esquemas Pydantic
├── security.py       # Hashing bcrypt + JWT
├── services.py       # Lógica de negocio y búsqueda
├── init_db.py        # Inicialización de BD + seed de traducciones
├── sincronizador.py  # Sincronización CassChoice (Fase A y B)
├── cass_client.py    # Cliente API CassChoice
├── index.html        # Frontend SPA B2B (vanilla JS)
├── routers/          # auth, catalogo, carrito, ordenes, ia, sincronizacion
├── requirements.txt
├── .env.example      # Plantilla de configuración
├── Dockerfile / docker-compose.yml
└── DEPLOYMENT_GUIDE.md # (este archivo)

```

Generado como parte del overhaul e-commerce B2B. Precios con margen incluido · Integración CassChoice.